GSOC 2026 Application RADIS Tanisha Dahale

Integrate a Modern Lazy-Loading Alternative for Large-Scale Spectroscopic Database Processing

Mentors: Nicolas Minesi *@minouHub*, Pratham Hole *@Prtm2110*

Google Summer of Code 2026 | OpenAstronomy

Name:	Tanisha Dahale
GitHub ID:	tanishadahale
Email:	tanishadahale2005@gmail.com
Project Size:	350 hours (Large)

Contents

1 Introduction **3**

2 Project Background **4**
2.1 The Scale Problem in Spectroscopic Databases . . . 4
2.2 RADIS Database Processing Architecture . . . 4
2.3 Current Vaex Integration: Scope Analysis . . . 5
2.4 Configuration System . . . 5

3 Evaluation of Alternatives **6**
3.1 Candidate Libraries . . . 6
3.2 Comparative Analysis . . . 6
3.3 Recommendation . . . 7

4 Technical Implementation Plan **8**
4.1 Architecture: Centralized Adapter Layer . . . 8
4.2 Vaex → Polars API Mapping . . . 9
4.3 Phase 1: Benchmarking Framework . . . 9
4.4 Phase 2: DataFileManager Extension . . . 9
4.5 Phase 3: IO Layer Migration . . . 10
4.6 Phase 4: Computation Pipeline Migration . . . 10
4.7 Phase 5: Backward Compatibility & Cache Migration . . . 10
4.8 Phase 6: Tests & Validation . . . 10
4.9 Secondary Goals . . . 10

5 Milestones & Timeline **11**
5.1 Milestone Definitions . . . 11
5.2 Detailed Timeline (350h, ∼12–13 weeks) . . . 11

6 Contributions to RADIS **13**
6.1 Merged Pull Requests (8) . . . 13
6.2 Open Pull Requests (3) . . . 13
6.3 Key Qualification Highlights . . . 13

7 About Me **15**
7.1 Background . . . 15
7.2 Why This Project . . . 15
7.3 Why OpenAstronomy . . . 15
7.4 Commitment . . . 16

8 References **16**

1. Introduction

RADIS is an open-source Python library for fast line-by-line calculation of high-resolution molecular spectra. It processes spectroscopic databases—HITRAN, HITEMP, ExoMol, and GEISA—containing millions to hundreds of millions of spectral lines for absorption and emission under various thermal conditions. For large databases such as HITEMP CO_2 (∼326 million transitions, 10–19 GB as HDF5), loading everything into RAM is impractical. RADIS adopted Vaex for its memory-mapping and lazy-loading capabilities, allowing data to stay on disk with only requested columns and rows loaded into memory.

However, Vaex is now effectively unmaintained. While version 4.19.0 exists, it does **not** support Python $\geq$ 3.13. RADIS already restricts Vaex to `python_version < "3.13"` in `pyproject.toml` (lines 88–91). The Polars May 2025 benchmarks dropped Vaex comparisons entirely due to lack of maintenance. Multiple RADIS issues document the growing Vaex problems:

Issue	Description
#658	Main issue requesting fast-memory-mapping alternatives to Vaex
#578	Vaex installation errors with Python >3.8
#338	Partition function calculations stuck in Spyder IDE
#746	Full Vaex implementation of broadening coefficients incomplete

Table 1: Open RADIS issues related to Vaex problems

This project proposes to replace Vaex with a modern, actively-maintained lazy-loading library. Based on thorough analysis, **Polars** is the recommended primary candidate due to its expression-based API (similar to Vaex), Apache Arrow memory model, and native Parquet support. The migration will preserve RADIS's ability to process large databases without loading into RAM, maintain backward compatibility with existing HDF5 caches, and enable future Parquet format support.

2. Project Background

2.1 The Scale Problem in Spectroscopic Databases

Spectroscopic databases vary enormously in scale. While HITRAN contains approximately 11 million lines (moderate), databases like HITEMP CO_2 contain ∼326 million transitions spanning 10–19 GB as HDF5, and ExoMol line lists for certain molecules exceed 10 billion transitions. A machine with 16 GB of RAM simply cannot load a 19 GB HDF5 file into memory.

Vaex solved this through memory-mapping (`mmap`), where data stays on disk and only requested columns or rows are loaded into RAM on demand. This approach is central to RADIS's ability to handle large databases, and any replacement must preserve this lazy-loading capability.

2.2 RADIS Database Processing Architecture

The data flow in RADIS follows a clear pipeline from raw database files through parsing, caching, and finally into the line-by-line (LBL) computation engine. Figure 1 illustrates this architecture.

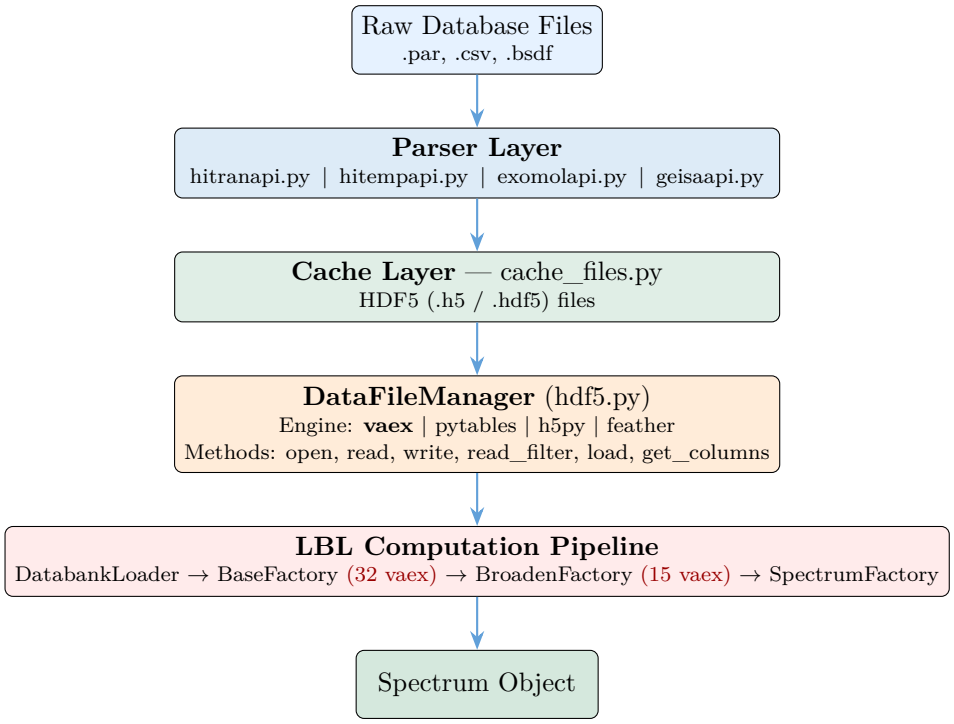

Figure 1: RADIS data flow pipeline from raw database files to Spectrum output. Red annotations indicate Vaex-dependent code that must be migrated.

2.3 Current Vaex Integration: Scope Analysis

A comprehensive audit of the RADIS codebase reveals Vaex usage across **23 files** with **90+ conditional branches**. Table 2 provides a detailed breakdown organized by subsystem.

File	Vaex Calls	Complexity	Category
`radis/lbl/base.py`	32	HIGH	Computation
`radis/lbl/broadening.py`	15	HIGH	Computation
`radis/api/hitranapi.py`	13	MED	Parser
`radis/api/hdf5.py`	10	HIGH	IO/Storage
`radis/api/exomolapi.py`	6+	HIGH	Parser
`radis/lbl/loader.py`	5	MED	Computation
`radis/api/dbmanager.py`	3	MED	IO/Storage
`radis/api/hitempapi.py`	2	LOW	Parser
`radis/levels/partfunc.py`	3	LOW	Utility
`radis/phys/convert.py`	2	LOW	Utility
`radis/api/tools.py`	2	LOW	Utility
`radis/misc/arrays.py`	1	LOW	Utility
`radis/misc/utils.py`	1	LOW	Utility
`radis/lbl/factory.py`	1	LOW	Config
`radis/lbl/calc.py`	1	LOW	Config
8 test files	various	MED	Testing
Total	**90+**		**23 files**

Table 2: Vaex usage heatmap across the RADIS codebase, color-coded by migration complexity.

2.4 Configuration System

Two keys in `default_radis.json` control engine selection. `MEMORY_MAPPING_ENGINE` (line 25) accepts `"auto"`, `"vaex"`, `"pytables"`, or `"feather"`, and is resolved via `get_auto_MEMORY_MAPPING_ENGINE()` in `api/dbmanager.py` (line 49). `DATAFRAME_ENGINE` (line 29) accepts `"pandas"` or `"vaex"` and sets `self.dataframe_type`, which is checked in the 90+ conditional branches throughout the LBL computation pipeline. Additionally, `AUTO_UPDATE_DATABASE` (line 19) triggers automatic conversion of old cache formats when engine preferences change.

3. Evaluation of Alternatives

3.1 Candidate Libraries

Three modern libraries were evaluated as potential Vaex replacements, each offering lazy evaluation and efficient memory handling.

Polars (Recommended): Polars provides true lazy evaluation through `scan_parquet()`, `scan_ipc()`, and `scan_csv()`. Built on the Apache Arrow memory model, it enables zero-copy NumPy/Pandas interop. It is actively maintained with monthly releases, has a Rust backend, and fully supports Python 3.13+. Its expression-based API is philosophically similar to Vaex, and it offers native Parquet support aligned with the secondary format migration goal. The main limitation is the absence of native HDF5 reading, which requires an h5py bridge—a straightforward implementation.

DuckDB: DuckDB offers SQL-based lazy evaluation and can query Parquet, CSV, and HDF5 without full loading. It excels at filtering operations that map well to RADIS's `read_filter()` pattern. However, the SQL paradigm requires significantly more refactoring of the expression-based code in `lbl/base.py` (32 branches) and `lbl/broadening.py` (15 branches).

Dask: Dask provides a Pandas-compatible API requiring the least refactoring, with native lazy evaluation and HDF5 support. However, it is slower than Polars for single-machine workloads, has higher memory overhead, and is designed for distributed computing—overkill for RADIS's single-machine use case.

3.2 Comparative Analysis

Criterion	Polars	DuckDB	Dask
Lazy evaluation	Native LazyFrame	SQL queries	Task graph
Memory model	Apache Arrow	Apache Arrow	Chunked Pandas
HDF5 support	Via h5py bridge	Via extension	Native
Parquet support	Native (excellent)	Native (excellent)	Via PyArrow
Python 3.13+	Yes	Yes	Yes
API similarity to Vaex	**High** (expression)	Low (SQL)	Medium (Pandas)
Single-machine speed	**Fastest** (3× Pandas)	Fast	Slower
RADIS refactoring effort	Medium	High	Low
Community adoption	Large, growing	Large, growing	Mature, stable

Table 3: Comprehensive comparison of candidate libraries for Vaex replacement.

3.3 Recommendation

Polars is Recommended

Polars is the optimal choice for RADIS because:

- **Expression-based API** maps cleanly to Vaex's API surface, minimizing refactoring across 90+ conditional branches
- **Fastest single-machine performance**—3× over Pandas and 2× over Vaex in published benchmarks
- **Native Parquet support** enables the secondary goal of format migration
- **Zero-copy Arrow interop** minimizes conversion overhead with NumPy, critical for the computation pipeline
- **Actively maintained** with monthly releases and a rapidly growing community ensuring long-term viability

4. Technical Implementation Plan

4.1 Architecture: Centralized Adapter Layer

Rather than replacing 90+ scattered `if/elif` blocks one by one, the project introduces `radis/api/lazy_adapter.py`—a centralized adapter that provides a unified interface for all lazy-loading operations. The existing `DataFileManager` handles IO; the adapter handles computation-layer operations. Figure 2 illustrates the before-and-after architecture.

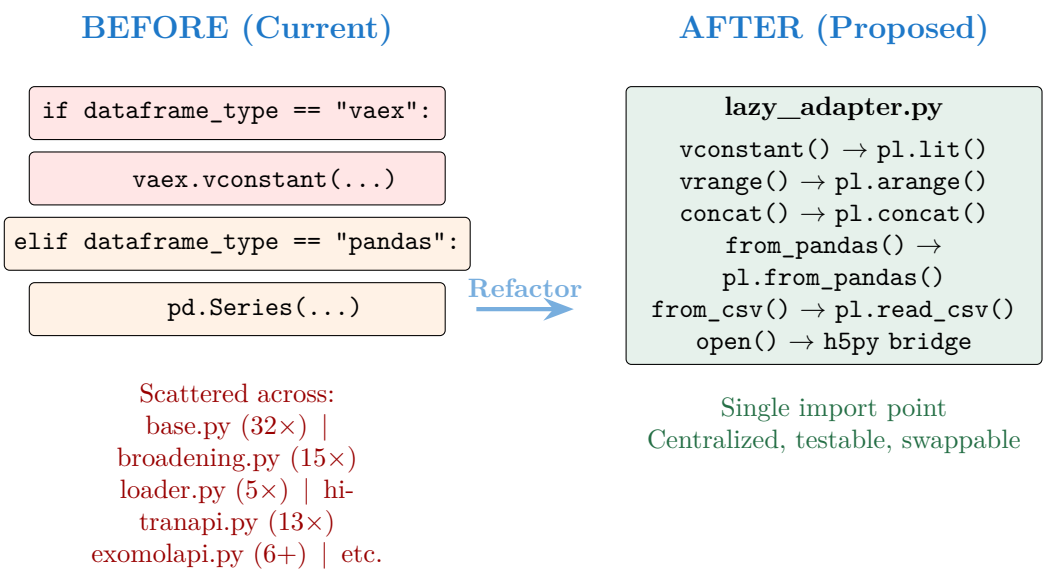

Figure 2: Before-and-after architecture: scattered conditionals replaced by a centralized adapter layer.

4.2 Vaex → Polars API Mapping

Table 4 provides the complete mapping from Vaex API calls to their Polars equivalents, covering all patterns found in the RADIS codebase.

Vaex API	Usage Context	Count	Polars Equivalent
`vaex.vconstant(v, n)`	Fill columns with NaN	∼30	`pl.lit(v).alias(name)`
`vaex.open(path)`	Open HDF5 files	∼6	h5py → `pl.DataFrame`
`vaex.from_pandas(df)`	Convert DataFrames	∼8	`pl.from_pandas(df)`
`vaex.from_csv(path)`	Read CSV files	∼4	`pl.scan_csv(path)`
`vaex.concat(dfs)`	Concatenate frames	∼5	`pl.concat(dfs)`
`vaex.from_arrays(**kw)`	Create from arrays	∼4	`pl.DataFrame(kw)`
`vaex.vrange(a, b)`	Range generation	∼3	`pl.arange(a, b)`
`df.export_hdf5()`	Write HDF5	∼5	`df.write_parquet()` / h5py
`df.to_pandas_df()`	Convert to Pandas	∼4	`df.to_pandas()`
`df.close()`	Release resources	∼6	No-op (Rust GC)
`df.length_unfiltered()`	Row count	∼25	`df.height` / `len(df)`
`df.column_names`	Column listing	∼4	`df.columns`
`df.extract()`	Materialize	∼2	`df.collect()`
`isinstance(..., Expression)`	Type check	∼4	`isinstance(..., pl.Expr)`
`col.countnan()`	NaN counting	∼5	`col.null_count()`

Table 4: Complete Vaex → Polars API mapping covering all patterns found in the RADIS codebase.

4.3 Phase 1: Benchmarking Framework

The project begins with a rigorous benchmarking phase extending the existing `test/benchmark/vaex_vs_pandas_performance.py`. Benchmarks will measure read speed, write speed, filter speed, peak memory, lazy evaluation overhead, CSV parsing, and concatenation across Polars, DuckDB, Dask, and Vaex. Critically, benchmarks will also compare **file formats**—HDF5 vs Parquet—to determine whether Parquet should be adopted as the default cache format from the start (rather than bridging HDF5 to Polars first). For filtered reads (RADIS's primary access pattern: loading spectral lines in a wavenumber range), Parquet with predicate pushdown may skip irrelevant row groups entirely, potentially outperforming HDF5 memory-mapping. Test databases include HITEMP OH (small), H2O (∼6 GB), CO_2 (full size at ∼19 GB), and large ExoMol molecules. Profiling tools include `memory_profiler`, `time`, and `tracemalloc`. The deliverable is a reproducible benchmark report with comparison charts, including a data-driven recommendation on file format.

4.4 Phase 2: DataFileManager Extension

The selected library will be implemented as a new engine in `radis/api/hdf5.py` (the `DataFileManager` class), supporting all core methods: `open()`, `read()`, `write()`, `read_filter()`, `load()`, `combine_temp_batch_files()`, `has_nan()`, `to_numpy()`, `add_metadata()`, `read_metadata()`, `get_columns()`, `guess_engine()`, and `cache_file()`. For Polars with HDF5, h5py will read HDF5 datasets into Arrow arrays, then convert to Polars DataFrames. For Polars with Parquet, native `pl.scan_parquet()` returns a LazyFrame where no data is read from disk until `.collect()` is called, with predicate pushdown that skips entire row groups outside the requested wavenumber range. The configuration in `default_radis.json` and `dbmanager.py` will be updated to prefer Polars when available.

4.5 Phase 3: IO Layer Migration

This phase systematically migrates all IO-layer files. In `exomolapi.py`, `vaex.from_csv()`, `vaex.from_arrays()`, and `vaex.vrange()` calls in `read_trans()`, `read_states()`, `pickup_gE()`, and broadening functions are replaced. In `hitempapi.py`, multi-file `vaex.from_pandas()` and `vaex.concat()` patterns are migrated. In `hitranapi.py`, all 13 quantum number string-slicing branches conditioned on `dataframe_type == "vaex"` are updated. Supporting files (`dbmanager.py`, `tools.py`, `io/query.py`) and utility files (`arrays.py`, `convert.py`, `utils.py`, `partfunc.py`) are also migrated. The function `update_pytables_to_vaex()` is generalized to `update_cache_format()`.

4.6 Phase 4: Computation Pipeline Migration

The largest phase addresses the LBL computation pipeline. In `lbl/base.py` (32 branches), patterns like `vaex.vconstant(np.nan, df.length_unfiltered())` become `df.with_columns(pl.lit(None).cast(pl.Float64).alias("col"))`. NaN counting, physical unit conversion (`cm2J_vaex()`), and expression arithmetic are all migrated. In `lbl/broadening.py` (15 branches), `vaex.expression.Expression` type checks become `pl.Expr` checks, and the `try/except isinstance` pattern is replaced with `pl.when().then().otherwise()`. The remaining files (`loader.py`, `factory.py`, `bands.py`, `calc.py`) and utility modules are updated accordingly.

4.7 Phase 5: Backward Compatibility & Cache Migration

Vaex HDF5 files use a column-based format under a `/table` group, which is readable via h5py. The generalized `update_cache_format()` function handles automatic conversion of old caches when `AUTO_UPDATE_DATABASE` is enabled. The `guess_engine()` function is extended to detect Polars-generated files, and `OLDEST_COMPATIBLE_VERSION` is bumped to reflect the new format.

4.8 Phase 6: Tests & Validation

All 8 test files with Vaex-specific code will be migrated. A comprehensive integration test suite for the new engine will be created, along with an updated multi-engine benchmark (replacing `vaex_vs_pandas_performance.py`). New tests cover cache migration and backward compatibility with old `.hdf5` files.

4.9 Secondary Goals

Parquet Format Support: Adding `"parquet"` as a `MEMORY_MAPPING_ENGINE` option enables Polars' native `pl.scan_parquet()` with predicate pushdown (filters pushed to row-group level, skipping data outside the wavenumber range without reading it) and column pruning (only requested columns read from disk). This offers potential 2–5× compression improvement over HDF5, with a new `.parquet` cache extension alongside `.h5`/`.hdf5`. Whether Parquet becomes the default format will be determined by the Phase 1 benchmarks.

GPU Acceleration Exploration: Documentation of potential GPU acceleration using `cudf` (NVIDIA RAPIDS) or the Polars GPU backend, identifying beneficial operations such as filtering, sorting, and broadening calculations. This is scoped as exploration and documentation only.

5. Milestones & Timeline

5.1 Milestone Definitions

Midterm Evaluation

Comprehensive benchmark report comparing Polars, DuckDB, Dask, and Vaex. Library confirmed with data-driven justification. DataFileManager extended with new engine support. IO layer migration complete (hdf5.py, exomolapi.py, hitempapi.py, hitranapi.py). LBL computation pipeline migration started (base.py, broadening.py).

Final Evaluation

Full Vaex replacement across all 23 files. LBL computation pipeline fully migrated (base.py, broadening.py, loader.py). User-selectable backend via `DATAFRAME_ENGINE` in `radis.json`. Backward compatibility verified with old caches. Parquet format support complete. Performance comparison report (before/after). Documentation and migration guide. All code merged with passing CI/CD.

5.2 Detailed Timeline (350h, ∼12–13 weeks)

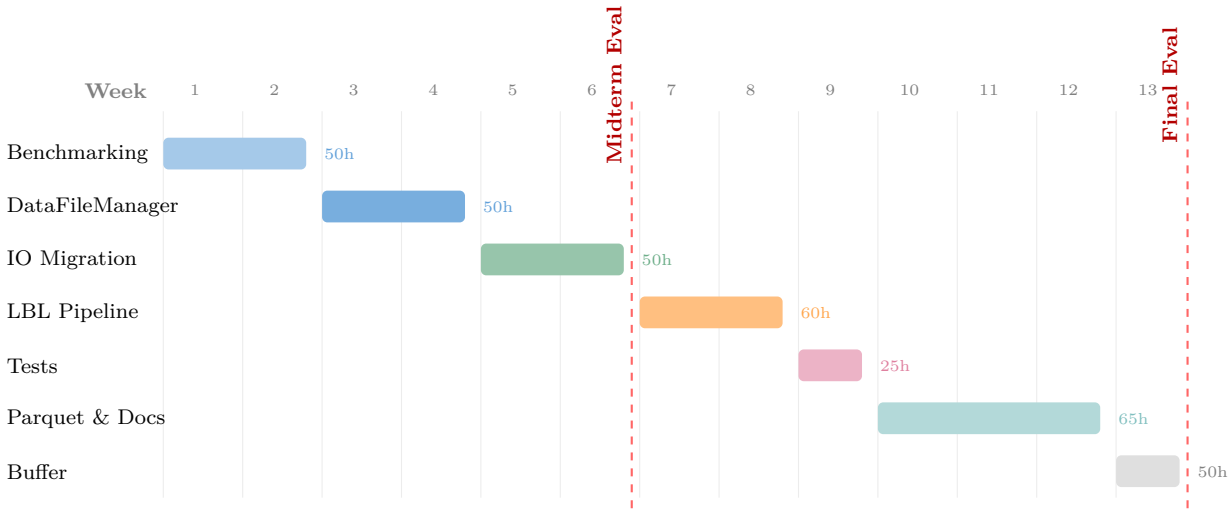

Figure 3: Gantt chart showing the 13-week timeline with evaluation milestones. Total: 350 hours with 50h buffer.

Pre-GSoC Period:
Continue contributing to RADIS (3 open PRs: #968, #967, #946). Deep study of Polars internals for HDF5 interop. Prototype basic Polars integration with DataFileManager.

Community Bonding (∼20h):
Architecture deep-dive with mentors. Download HITEMP databases for local benchmarking. Discuss methodology and finalize evaluation framework.

Weeks 1–2: Benchmarking (50h):
Extend existing benchmark suite to include Polars, DuckDB, and Dask. Benchmark both HDF5

and Parquet file formats to determine optimal cache format. Test on HITEMP OH, H2O, CO_2 and ExoMol databases. Produce comparison report with charts. Final library and format selection with mentors.

Weeks 3–4: DataFileManager & Foundation (50h):
Implement Polars engine in DataFileManager with all core methods. Build h5py bridge for HDF5→Polars conversion. Implement adapter layer and update configuration.

Weeks 5–6: IO Layer Migration (50h):
Migrate all parser APIs (exomolapi.py, hitempapi.py, hitranapi.py), database management files, and utility modules. Generalize cache format functions.

— Midterm Evaluation —

Weeks 7–8: LBL Computation Pipeline (60h):
Migrate the core computation files: base.py (32 branches), broadening.py (15 branches), loader.py (5 branches), and supporting factory/config files.

Week 9: Test Migration (25h):
Migrate 8 test files. Create integration tests for new engine. Backward compatibility tests with existing caches.

Weeks 10–11: Parquet Format & Dependency Cleanup (65h):
Implement Parquet engine in DataFileManager. Benchmark Parquet vs HDF5 for RADIS workloads. Update pyproject.toml and environment.yml. Remove Vaex dependency.

Weeks 12–13: Documentation & Polish + Buffer (50h):
User and developer guides. Performance comparison report (before/after). Update examples. Final CI/CD across Python 3.10–3.13. Buffer time for unexpected issues, edge cases, and CI failures.

— Final Evaluation —

6. Contributions to RADIS

Summary: 11 PRs — 8 Merged, 3 Open

My contributions span code quality, IO layer modifications, database API standardization, feature development, and test infrastructure—demonstrating breadth across the exact subsystems targeted by this project.

6.1 Merged Pull Requests (8)

PR	Title	Relevance to This Project
#936	Refactor: os.path → pathlib (partial)	**Systematic codebase-wide migration**—directly analogous to Vaex→Polars
#930	File-like object support for spectrum.store()	**IO layer modification**, file handling architecture
#914	Wildcard/list support for calc_spectrum databank	Feature development, calc_spectrum core API
#910	Update non-regression test values for HITRAN2024	Database handling, test infrastructure
#907	Automatic residual plotting for SpecDatabase.fit_spectrum	Spectrum analysis, SpecDatabase
#899	Standardize database progress across all APIs	**All 4 parser APIs**—exact files needing migration
#957	Fix LineSurvey to respect lineinfo	Tooltip improvements, formatting
#956	Fix TODOs: error/warning messages, bare except	Code quality across multiple modules

Table 5: Merged PRs with relevance to the proposed Vaex replacement project.

6.2 Open Pull Requests (3)

PR	Title	Status
#968	Add normalize support to get_distance()	Pending review
#967	Resolve TODOs: add missing warning, docstring	Pending review
#946	Fix #440: handle slit_dispersion wavespace change	Open

Table 6: Currently open PRs under review.

6.3 Key Qualification Highlights

PR #936 (pathlib refactoring) proves I can systematically migrate patterns across the entire codebase. This PR required identifying every `os.path` usage, understanding the context, choosing the correct `pathlib` equivalent, and verifying correctness—the exact same workflow needed for Vaex→Polars migration across 90+ branches.

PR #899 (database progress standardization) shows I understand all 4 database parser APIs (HITRAN, HITEMP, ExoMol, GEISA). These are the exact files requiring Vaex replacement, and I have already modified each one.

PR #930 (file-like objects) demonstrates IO layer modification experience. Understanding how RADIS handles file operations is essential for implementing the h5py bridge and new cache formats.

PR #914 (wildcard/list support) shows feature development integrating with `calc_spectrum`—RADIS's core user-facing API that must continue working seamlessly after the migration.

7. About Me

7.1 Background

I am a 3rd year AI & Data Science student with practical experience in Python, Pandas, Polars, DuckDB, Dask, NumPy, h5py, and scientific computing. My understanding of HDF5 and Parquet file formats, combined with experience in memory-efficient data processing and performance profiling, directly aligns with this project's requirements.

I have been contributing to RADIS since December 2025, accumulating deep familiarity with the codebase architecture. The pathlib refactoring (PR #936) taught me how to plan and execute systematic codebase-wide migrations—the defining skill for this project. Working across all 4 database APIs (PR #899) gave me familiarity with exactly the files that need Vaex replacement.

7.2 Why This Project

This project sits at the intersection of my strongest technical interests: data engineering, performance optimization, and scientific computing. My practical experience with Polars, DuckDB, and Dask means I can hit the ground running on both benchmarking and implementation without a steep learning curve.

Beyond technical fit, this project addresses a **critical infrastructure problem**. Vaex's inability to support Python ≥3.13 means RADIS will become increasingly difficult to install and maintain. Every new Python release widens this gap. The migration is not optional—it is necessary for RADIS's survival as a modern scientific tool. By completing this work, I will directly unblock researchers in plasma physics, atmospheric science, and astrophysics who depend on RADIS for spectroscopic calculations.

I am also drawn to this project because it demands the exact skill I have demonstrated most clearly in my contributions: **systematic, codebase-wide refactoring with zero regression**. My pathlib migration (PR #936) required the same discipline—audit every usage, understand context, choose the correct replacement, and verify correctness file by file. The Vaex→Polars migration is a larger-scale version of the same workflow, and I am confident in my ability to execute it methodically.

The Parquet format support is particularly exciting. Polars' native Parquet support with predicate pushdown could transform how RADIS handles large databases, enabling researchers to work with datasets that currently push the limits of available RAM.

7.3 Why OpenAstronomy

Open-source scientific computing combines my core interests in programming and applied science. OpenAstronomy's mission—making astronomy and astrophysics tools accessible, reproducible, and community-driven—resonates deeply with my belief that research software should be open, well-maintained, and usable by everyone.

RADIS's community has been exceptionally welcoming. From my first PR, mentors provided thoughtful, detailed code reviews that improved both the code and my understanding of the architecture. The Slack community is active and responsive, and the collaborative culture encourages contributions at all experience levels. This is the kind of open-source environment where I do my best work.

Contributing to tools used by active researchers provides a tangible sense of impact that personal projects cannot match. When a researcher uses RADIS to simulate a plasma spectrum or

analyze atmospheric data, the code I write directly supports their scientific discovery. This connection between engineering work and real-world outcomes is what motivates me to continue contributing to OpenAstronomy projects well beyond the GSoC period.

I plan to remain an active RADIS contributor after GSoC, helping maintain the Polars backend, reviewing community PRs, and potentially mentoring future GSoC students on the codebase I will have helped modernize.

7.4 Commitment

I will dedicate **350 hours** (∼30 hours/week) to this project. I will maintain weekly syncs with mentors @Prtm2110 and @minouHub, follow a PR-based workflow with incremental reviews, and remain available for communication across timezones. My academic schedule has been arranged to accommodate this commitment, and I will continue contributing to RADIS beyond the GSoC period.

Item	Details
Laptop	Windows 11 64-bit
RAM	16 GB
IDE	VS Code, PyCharm
Technical Skills	Python, Pandas, Polars, DuckDB, Dask, NumPy, h5py, Git
File Formats	HDF5, Parquet, Arrow IPC

Table 7: Development platform and technical skills.

8. References

[1] Pannier, E., & Laux, C. O. (2019). RADIS: A nonequilibrium line-by-line radiative code for CO_2 and HITRAN-like database species. *JQSRT*, 222, 12–25.
https://doi.org/10.1016/j.jqsrt.2018.09.027

[2] Vaex GitHub repository (Python 3.13 not supported).
https://github.com/vaexio/vaex

[3] Polars documentation.
https://pola.rs/

[4] DuckDB documentation.
https://duckdb.org/

[5] RADIS Issue #658 — "Add fast-memory-mapping alternative(s) to vaex."
https://github.com/radis/radis/issues/658

[6] RADIS Issue #578 — Vaex installation errors.
https://github.com/radis/radis/issues/578

[7] RADIS Issue #338 — Vaex stuck in Spyder IDE.
https://github.com/radis/radis/issues/338

[8] Apache Arrow specification.
https://arrow.apache.org/

[9] Apache Parquet specification.
https://parquet.apache.org/

[10] Gordon, I. E., et al. (2022). The HITRAN2020 molecular spectroscopic database. *JQSRT*, 277, 107949.
https://doi.org/10.1016/j.jqsrt.2021.107949

[11] Polars PDS-H Benchmarks (May 2025) — Vaex removed from comparisons due to lack of maintenance.
https://pola.rs/posts/benchmarks/